

Reviewer Pack — Minimal Order of Quasi-symmetric Designs and Resolution Proo...

Entry 07a8996922c9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 23:33:57 UTC

Minimal Order of Quasi-symmetric Designs and Resolution Proof Width Correlation

Entry ID: 07a8996922c9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 23:33:57 UTC

1. Conjecture

Field A (mathematical branch): Quasi-group Theory (Quasi-symmetric Designs) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every instance φ of a Boolean formula, the resolution proof width $w(\varphi)$ is linearly correlated with the minimal order of a quasi-symmetric design D associated with φ such that $w(\varphi) = \Theta(|D|)$. Additionally, $|D| \leq \Omega(n^2)$, where n is the number of variables in φ .

Rationale (proposer's reasoning):

Quasi-symmetric designs encode symmetries in non-trivial ways and have not been extensively applied in computational complexity theory. Their minimal order could potentially reveal underlying structural properties that are relevant to proof complexity, offering a new perspective on resolution proofs.

Taxonomy category: Quasi_group_theory_resolution_proof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ce388821cc05647c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The Pearson correlation coefficient between the minimal order of quasi-symmetric designs $|D|$ and resolution proof width $w(\varphi)$ for each instance φ must be ≥ 0.8 , with no seed producing a correlation coefficient < 0.8 . Additionally, the mean correlation across all seeds must be ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.90	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_formula(n):
        # Generate a random Boolean formula with n variables and 2n clauses
        variables = [f'x{i}' for i in range(n)]
        clauses = []
        for _ in range(2 * n):
            clause = random.sample(variables + [f'~{v}' for v in variables], k=3)
            clauses.append(' or '.join(clause))
        return ' and '.join(clauses)

    def dpll_solver(cnf, assignment={}):
        if not cnf:
            return True
        unit_clauses = [c for c in cnf if len(c) == 1]
        if unit_clauses:
            literal = unit_clauses[0][0]
            var = literal.strip('~')
            value = literal.startswith('~') ^ (var in assignment and assignment[var])
            if not dpll_solver([c for c in cnf if literal not in c], {**assignment, var: value}):
                return False
        pure_literals = [l for l in set(''.join(cnf)) if l != '~' and f'~{l}' not in ''.join(cnf)]
        if pure_literals:
            literal = pure_literals[0]
            var = literal.strip('~')
            value = literal.startswith('~') ^ (var in assignment and assignment[var])
            if not dpll_solver([c for c in cnf if literal not in c], {**assignment, var: value}):
                return False
        literals = [l for l in set(''.join(cnf)) if l != '~' and f'~{l}' not in ''.join(cnf)]
        literal = random.choice(literals)
        var = literal.strip('~')
        value = literal.startswith('~') ^ (var in assignment and assignment[var])
```

```

    return dpll_solver([c for c in cnf if literal not in c], {**assignment, var: value}) or \
        dpll_solver([c for c in cnf if f'~{literal}' not in c], {**assignment, var: not val

def resolution_proof_width(cnf):
    # Simple heuristic to estimate the width of a resolution proof
    return max(len(set(l.split(' or '))) for l in cnf)

n = random.randint(5, 40)
formula = generate_boolean_formula(n)
cnf = [c.strip().split(' or ') for c in formula.split(' and ')]

if not dpll_solver(cnf):
    return {
        "metric_name": "resolution_proof_width",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Formula is unsatisfiable"
    }

width = resolution_proof_width(cnf)
return {
    "metric_name": "resolution_proof_width",
    "metric_value": width,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_dev = math.sqrt(sum((r["metric_value"] - mean_width) ** 2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_dev} support_fraction={support_fra
    elif any(not r["conjecture_holds"] and r["metric_value"] < 0.8 for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if not r["conjecture_holds"] an
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient<0.8\" first_failing_see
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d903b598.py", line 88, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d903b598.py", line 62, in run_trial
    if not dpll_solver(cnf):
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d903b598.py", line 40, in dpll_solver
    pure_literals = [l for l in set(''.join(cnf)) if l != '~' and f'~{l}' not in ''.join(cnf)]
                                ^^^^^^^^^^^^^
TypeError: sequence item 0: expected str instance, list found
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to ensure it can complete without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14526
2	preregistration	ollama_remote	glm4:latest	0	0	17218
3	novelty	ollama_remote	glm4:latest	0	0	15545
4	novelty	ollama_remote	glm4:latest	0	0	9051
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17670
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16541
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11148
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12523
9	judge	ollama_remote	glm4:latest	0	0	8688

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122909 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/07a8996922c9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/07a8996922c9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/07a8996922c9.tar.gz` (if generated)